ITcon

www.itcon.org - Journal of Information Technology in Construction - ISSN 1874-4753

INVENTING ONTOLOGY OF DESIGN GRAMMARS AS A SEMANTIC CORE FOR INTERACTIVE DATA-DRIVEN DESIGN VALIDATION

SUBMITTED: June 2026

REVISED:

PUBLISHED:

EDITOR: Evgenii Ermolenko

DOI:

Evgenii Ermolenko, Mr,
University of Minho; EAAD; Lab2PT;ISISE, Guimaraes, Portugal

ORCID: 0009-0003-1228-9550
email erarchitech@gmail.com

Bruno Figueiredo, Pr,
University of Minho; EAAD; Lab2PT;IN2PAST, Guimaraes, Portugal;

ORCID: 0000-0001-8439-7065
email bfigueiredo@eaad.uminho.pt

Miguel Azenha, Pr,
University of Minho; ISISE; ARISE; Department of Civil Engineering, Guimaraes, Portugal;

ORCID: 0000-0003-1374-9427
email miguel.azenha@civil.uminho.pt

SUMMARY: Design grammars offer a formal language for describing spatial and regulatory constraints in architectural design, yet their encoding in machine-readable ontologies remains fragmented and ad hoc. This paper presents the development of a structured ontology for design grammars, intended to act as a semantic core for data-driven design that bridges abstract design rules, parametric design definitions, and object-oriented model representations within Parametric Design and Building Information Modelling (BIM) interoperable workflows. Aside from existing IFC-schema-based validators, this research shifts focus from file-based compliance checking to object-level communication and validation of design data integrated into interactive workflows. Rather than describing a complete software system, the paper concentrates on the ontology itself: its development methodology, conceptual structure, and formalization. Following established ontology-engineering activities - specification, conceptualization, formalization, and implementation - the ontology reuses and aligns with existing design and topology ontologies (Gero's Function–Behaviour–Structure model, Design Criteria Modelling, the Building Topology Ontology, and Topologic). The schema employs OWL 2 constructs and the Semantic Web Rule Language (SWRL) to represent architectural design grammars as typed rule nodes within a property-graph (LPG) database. The ontology is organized as a cross-cutting Core band - grounded in the Function–Behaviour–Structure framework - over five co-resident graph layers: an OntoGraph of domain vocabulary, a Metagraph that decomposes each rule into atomic predicates (class atoms, datatype-property atoms, and SWRL built-in atoms), a ValidationGraph that records execution and results, a SpecGraph for shared knowledge, and a ComputationGraph that mirrors the parametric design model and maps directly onto Grasshopper definitions. A violation-pattern semantics encodes each constraint so that a rule fires precisely when the constraint is broken, and a cross-layer bridge attaches rule atoms to the parameters of the parametric definition that produces the validated geometry. The ontology is delivered as a vendor-neutral core module with three optional alignment extensions - standards, BOT, and Topologic - following the W3C core-plus-extension pattern. Project-scoped isolation allows multiple design projects to coexist within a single graph while still enabling knowledge sharing, so the ontology can accumulate lessons learned across a shared stakeholder memory. Three use cases - natural-language design-rule communication, master-plan design-state validation, and encoded spatial-programme implementation in

parametric plan-layout generation - are formalized to illustrate the ontology's applicability potential; their operational realization within a full authoring-and-validation framework is left to a further stage of research. The contribution advances the state of the art by providing a scalable, technology-agnostic ontology schema for design grammars that is compatible with mainstream BIM authoring and computational design tools and accessible to practitioners without formal ontology expertise.

KEYWORDS: *design grammars; ontology; SWRL; knowledge graph; modular ontology; Grasshopper;* design rules; BIM; parametric design

REFERENCE: *First N. Lastname & First N. Lastname (2023). Article title. Journal of Information Technology in Construction (ITcon), Vol. 0, pg. 1-2, DOI: 10.36680/j.itcon.2023.0*

1. INTRODUCTION

The built environment is governed by an increasingly dense body of regulatory codes, planning guidelines, and project-specific design rules that architects and engineers must navigate throughout the design process. Manual compliance validation is time-consuming, error-prone, and difficult to scale across large project portfolios and design stages. The emergence of Building Information Modelling (BIM) as the dominant digital medium for architectural design has created an opportunity to automate compliance checking by querying structured geometric and semantic data against machine-readable rule sets (Eastman et al., 2009).

Design grammars, first introduced by Stiny and Gips (1972) as shape grammars, provide a generative, rule-based formalism for describing spatial configurations. Subsequent extensions - including parametric shape grammars, description-logic grammars, and ontology-driven rule systems - have sought to bring this formalism closer to practical compliance-checking workflows (Duarte, 2005; Beirão et al., 2011). However, the translation of design-grammar rules into executable ontology representations that operate directly on BIM data remains largely unsolved. A central obstacle is the absence of a shared, well-structured ontology that can host such rules: existing efforts tend to author ontologies ad hoc for individual studies, with little attention to reuse, alignment, or systematic development. This paper addresses that gap by focusing on the ontology itself rather than on any particular software system built around it.

An ontology functions as the semantic core - effectively the “brain” - of a knowledge graph: it interprets entities and infers semantic relationships in response to queries posed to the system. To organize and connect concepts that originate in diverse domains of architectural knowledge, a common and shared understanding of those concepts is required. Elaborating an ontology that embraces and extends existing design domains and their interrelationships therefore establishes the shared semantic core on which true design-grammar reasoning can be built.

Ontology-based compliance checking has been studied in the AEC sector for two decades, from early rule systems operating on enriched IFC files (Eastman et al., 2009) to OWL-based approaches using IfcOWL, SPARQL, and SWRL (Pauwels & Terkaj, 2016; Pauwels et al., 2017), and more recently to knowledge-graph backends for rule storage and evaluation (Zhou et al., 2022). Property-graph databases such as Neo4j have likewise been adopted as flexible stores for heterogeneous building knowledge (Sacks et al., 2020). The present work builds on this lineage furhter gaining to develop common ontology that such validation systems presuppose extending on cross-platform and cross-format basis. Consequently, within the research scope, the ontology described here forms the semantic core of the broader cross-platform knowledge-management framework introduced in Ermolenko et al. (2026).

Against this background, the contributions of this paper are: (1) the application of a systematic ontology-development methodology to design grammars within a cross-domain knowledge system; (2) a five-layer ontology schema, organized as a cross-cutting Core band over an OntoGraph, a Metagraph, a ValidationGraph, a SpecGraph, and a ComputationGraph, with a violation-pattern SWRL encoding and a cross-layer bridge that links rule atoms to parametric-design parameters; (3) a modular packaging of the ontology as a vendor-neutral core with three optional standards, BOT, and Topologic alignment extensions, together with a worked mapping between a Grasshopper parametric definition and the ComputationGraph; and (4) three formalized use cases that illustrate the ontology’s applicability potential. The remainder of the paper is organized as follows. Section 2 presents the ontology-development methodology and the specification of the design-grammar domain, including the reuse of existing ontologies. Section 3 describes the resulting design-grammar ontology - its Core band and five-layer modular schema, node and relationship types, SWRL encoding, and the mapping of parametric definitions onto the ComputationGraph. Section 4 evaluates the ontology through three formalized use cases. Section 5 discusses expressiveness, limitations, and positioning relative to related approaches. Section 6 concludes. The operationalization of the ontology within a complete authoring-and-validation framework - including the natural-language rule-encoding pipeline - is the subject of a companion paper.

2. ONTOLOGY DEVELOPMENT METHODOLOGY

2.1 Development Activities

There is no single correct way to model a domain; viable alternatives always exist (Noy & McGuinness, 2001). Accordingly, the design-grammar ontology was developed following a structured set of ontology-development activities widely used in ontological engineering (Sack, 2019), comprising four principal stages organized as an iterative process across increasing levels of formality. Specification establishes the main definition of the ontology. Conceptualization structures domain knowledge into a conceptual model. Formalization expresses that conceptual model in a (semi-)computable form. Implementation constructs a computable model in an ontology-representation environment. These principal activities are accompanied by supporting activities carried out continuously throughout development: knowledge acquisition (gathering knowledge from experts and the literature), evaluation (technical assessment of the ontology at each step), reuse (merging and aligning existing ontologies), documentation, and configuration management (version control).

2.2 Specification: Domain, Scope, and Feasibility

The specification stage fixes the boundaries and purpose of the ontology. The domain is procedural design in the conceptual stage of architecture, and the scope is design grammars - problem formulation and parametric schema, design inputs, constraints, and relationships. The purpose is to support design-system decomposition and grammar inference, and to carry design intent into later stages of parametric design and grammar validation. The intended end-users are architects and clients, while maintainers are BIM departments. The anticipated benefits are rigorous communication of design grammars, simplified management of complex design ideas and multiple constraints, automatic design-intent validation, and improved sustainability of design knowledge across reformulations and reuse.

The scope is informed by how architects reason: through problem–solution co-evolution (Schön, 1992; Poon & Maher, 1997; Dorst & Cross, 2001), iterative divergence and convergence with problem reformulation, framing and abductive reasoning (Dorst, 2011), analogical reasoning such as design-by-analogy and case-based reasoning (Gero, 1992; Mubarak, 2004; Fu et al., 2015), and function–structure mapping and decomposition (Gero, 1990). From these, the ontology is required to support a corresponding set of tasks: decomposing the design problem into computable parts; enabling a modular structure of design knowledge; composing complex algorithms within a single class and its properties; raising interoperability and shared perception across stakeholders; translating design knowledge among authoring systems; enabling a semantic reasoner over defined design rules; and enabling automatic design-intent validation.

Feasibility is assessed through the competency questions the ontology must answer, which both bound its scope and provide acceptance criteria for evaluation. Representative competency questions include: Does a staircase comply with defined dimension constraints and spatial adjacencies? Does the current model state meet the lighting constraints? Can a design space of plan configurations be generated according to the spatial programme and constraints? Does the detailed-design BIM model comply with the initial parametric schema defined at the conceptual stage? The three use cases of Section 4 are derived directly from these questions.

2.3 Knowledge Reuse and Alignment

A core principle of the methodology is reuse of existing ontologies rather than de novo authoring. Four sources were analyzed for versatility of implementation, accessibility for external use, and maintainability, and were aligned to the design-grammar domain while preserving a common SWRL convention. Following the W3C core-plus-extension pattern, each external alignment is kept out of the vendor-neutral core ontology and confined to a dedicated extension module, so that reuse never couples the core schema to a particular vocabulary or platform. The Function–Behaviour–Structure framework supplies the conceptual scaffolding for the cross-cutting Core band, Design Criteria Modelling informs both the rule encoding and the ComputationGraph layer, and the topology ontologies are bridged through separate BOT and Topologic extension modules.

The first source is the Design Knowledge Ontology grounded in Gero's Function–Behaviour–Structure (FBS) framework (Gero, 1990; Gero & Kannengiesser, 2004), which decomposes a design into functions mapped to structures, with behaviour mediating the relationship and supporting reformulation. FBS provides modular, transparent, and domain-agnostic design reasoning - for example, designing a chair decomposes the function "seating" into expected behaviours (load capacity, comfort, light weight) realized by alternative structures, with reformulation driven by the gap between expected and actual behaviour.

The second source is Design Criteria Modelling (DCM) (Lin, 2021), which distinguishes three types of design information - semantic, topological, and geometric - and uses SWRL expressions of the form Body → Head to associate a semantic ontology with algorithmically computed topological relationships such as separated, connective, adjacent, and enclosing. DCM directly informs both the rule encoding adopted here and the ComputationGraph layer (Section 3), in which the algorithmically computed quantities that DCM evaluates are themselves modelled as parameters of a parametric design definition. For instance, DCM expresses a vision-quality criterion as:

IndoorSpace(?s) ∧ Window(?w) ∧ hasWindow(?s, ?w) ∧ OutdoorSpace(?x) ∧ LandscapeElement(?y) ∧ hasQuality(?y, "Good") ∧ isAdjacent(?s, ?x) ∧ FaceTo(?w, ?y) → hasVisionQuality(?s, "Good")

This pattern - semantic atoms combined with algorithmically computed topological predicates - is the conceptual antecedent of the encoding formalized in Section 3.

The third source comprises topology ontologies: the Building Topology Ontology (BOT) (Rasmussen et al., 2017, 2020) and the Topologic ontology (Jabi, 2021), which provide navigable representations of building topology for spatial relationships. Rather than embedding these vocabularies in the core schema, the ontology aligns to them through two dedicated extension modules: a BOT extension that maps the Core topology concept onto BOT's building/storey/space decomposition, and a Topologic extension that bridges the geometric output of the ComputationGraph to Topologic's hierarchical decomposition (cluster, cell-complex, cell, shell, face, wire, edge, vertex). Complementary shape-grammar and building-product vocabularies were additionally considered for terminology alignment. Across all sources, the violation-pattern SWRL convention drawn from DCM is retained, ensuring that reused concepts integrate consistently into the design-grammar schema.

2.4 Formalization and Implementation Environment

In the formalization stage the conceptual model is expressed using OWL 2 constructs and SWRL Horn clauses (Section 3). In the implementation stage the ontology is realized as a property graph in Neo4j, with the conceptual model optionally authored in a dedicated ontology editor such as Protégé. The ontology is designed to operate within mainstream BIM and parametric-design environments - BIM authoring and parametric tools (e.g., Rhinoceros, Revit) and visual-programming editors (e.g., Grasshopper, Dynamo). This paper formalizes the schema at the level of node labels, relationships, and SWRL encoding; the surrounding software services that exercise the ontology are out of scope and are addressed in the farther research stage.

3. THE DESIGN GRAMMAR ONTOLOGY

The ontology is organized as a cross-cutting Core band over five co-resident graph layers within a single property-graph (Neo4j) database instance, sharing nodes through typed relationships. The Core band holds the over-layer concepts that every layer references - the Function–Behaviour–Structure roles (Object, Function, Behaviour, Structure), Geometry and Topology, the captured design states (ObjectState and DefState), and a unified interaction Session. The OntoGraph layer captures the ontological vocabulary - domain classes, datatype properties, object properties, and SWRL built-ins. The Metagraph layer captures the structural decomposition of each design rule into rule, atom, variable, and literal nodes. The ValidationGraph layer records the execution of rules against a design state and the resulting per-object outcomes. The SpecGraph layer holds shared notes and tags that accumulate project knowledge. The ComputationGraph layer models the parametric design definition that produces the geometry under validation, and maps directly onto Grasshopper definitions (Section 3.5). The five layers are partitioned logically by a graph property rather than physically, so cross-layer relationships - for example, a Metagraph atom that constrains a ComputationGraph parameter - can be traversed without federation.

3.1 Ontology Architecture: Core Band and Five-Layer Modular Schema

The Core band sits above the five layers and is intentionally not tagged to any one of them, because its concepts are referenced from several layers at once. It encodes the Function–Behaviour–Structure (FBS) reading of a design artefact: an Object carries a Function (the design intent that rules enforce), is realized through Behaviour (the computational process that generates it), and yields a Structure (its geometric and topological output). Geometry and Topology are first-class Core concepts so that both the validated model and the parametric

definition can refer to the same spatial vocabulary. Two design-state concepts - ObjectState, a snapshot of an object's bound variables, and DefState, a snapshot of the parametric definition - capture the state that a validation run consumes, and a single Session concept records each authoring, query, or validation interaction. Keeping these over-layer concepts in one band is what lets the Metagraph, ValidationGraph, and ComputationGraph remain decoupled yet interoperable.

The OntoGraph corresponds conceptually to a lightweight OWL 2 ontology in which domain entities (e.g., Building, Floor, UrbanBlock) are represented as Class nodes and their measurable attributes (e.g., hasHeightM, grossFloorArea) are represented as DatatypeProperty nodes with typed ranges (xsd:decimal, xsd:integer, xsd:boolean). Object properties capture relationships between domain entities (e.g., containedIn, adjacentTo). All OntoGraph nodes are tagged with graph = 'OntoGraph' and a project identifier, enabling multiple projects to share a single database instance while maintaining logical isolation.

The Metagraph layer instantiates rules as Rule nodes, each identified by a structured Rule_Id of the form R_<DOMAIN>_<PROPERTY>_<LIMIT>_V (e.g., R_URB_HEIGHT_MAX_75_V). Each Rule node is connected to a set of Atom nodes via HAS_BODY (antecedent) and HAS_HEAD (consequent) relationships, ordered by an integer pos attribute. Atom nodes carry typed references (REFERS_TO) to ontology terms in the OntoGraph and argument relationships (ARG) to Var and Literal nodes. Rules are grouped into a RuleSet for evaluation. This decomposition makes every component of a SWRL rule individually addressable and queryable, which is a prerequisite for inspecting, auditing, and updating rules at the level of individual atoms.

The ValidationGraph layer records what happens when a RuleSet is evaluated against a design state. A ValidationRun captures a single evaluation - its rule set, the design state consumed, a timestamp, and a status - and is linked to one ValidationEntity per checked object, each recording the object instance, the property and value observed, and a pass or fail outcome. An ObjectInstance is the concrete geometric thing under test; it carries an ObjectState and a GeoRef, a neutral handle to the externally stored geometry held in the source viewer or model platform. A Reasoner node represents the evaluating component (the Grasshopper "Validator"), and an IntegrationConfig records the generic external project and model-version identifiers needed to fetch geometry, without naming a specific product. This layer aligns conceptually with the W3C PROV-O, SOSA, SHACL, and GeoSPARQL vocabularies, an alignment that is declared in a separate extension module rather than the core (Section 3.4).

The SpecGraph layer holds shared project knowledge as lightweight notes and tags. A SpecNote is a titled text artefact with a source; a SpecTag is a controlled tag; and a SpecClass groups instances of a given type. This layer lets the system accumulate lessons learned - decisions, debugging notes, and conventions - alongside the formal rules, and aligns with the SKOS and Dublin Core vocabularies through the standards extension.

The ComputationGraph layer is the fifth and most recent layer. It models the parametric design definition that generates the geometry a rule checks, expressed in the FBS terms of the Core band: the definition is the Behaviour that turns Function into Structure. Its central hierarchy is Algorithm → Procedure → Pattern, which corresponds one-to-one to a Grasshopper definition, its clusters, and the components inside them. Parameters (constant, variable, or emergent) are the inputs and outputs of these elements, exposed through typed Interface nodes. The layer is connected to the rule structure by a single cross-layer bridge, attributeOf, which links a Metagraph atom to the ComputationGraph Parameter it constrains; this bridge is what makes a design rule and the parametric definition that must satisfy it mutually addressable, and is the basis for the bidirectional translation between design grammars and parametric definitions developed in Section 3.5.

3.2 SWRL Rule Encoding and Violation-Pattern Semantics

Design rules are encoded as SWRL Horn clauses of the form Body → Head, where the body atoms express the condition that violates the constraint and the head atom asserts a boolean violation flag. This inverted, violation-first semantics simplifies rule evaluation: a rule fires if and only if the constraint is broken, avoiding the need to reason over the complement of a positive condition. For example, the regulation "maximum building height is 75 metres" is encoded as:

$$Building(?b) \wedge hasHeightM(?b, ?h) \wedge swrlb:greaterThan(?h, 75.0) \rightarrow violatesMaxHeight(?b, true)$$

Minimum constraints use swrlb:lessThan, equality constraints use swrlb:notEqual, and range constraints are decomposed into two separate violation rules - one for the lower bound and one for the upper bound - to maintain the principle that each Rule node encodes a single, atomic constraint. This rule-separation principle avoids compound rules that are difficult to trace during debugging and audit.

The violation-pattern semantics is applied consistently across all rule types supported by the schema. It operates under an open-world assumption: if no Building node is present in the graph, a height rule will not fire, regardless of whether the constraint is semantically satisfied. Section 5.1 discusses this limitation and its mitigation.

3.3 Graph Schema: Node Labels and Relationship Types

Table 1 summarizes the node labels, key properties, and graph-layer assignments of the schema. Table 2 lists the allowed relationship types and their source–target constraints. Together, these tables constitute the normative schema definition against which encoded rules are validated before they are committed to the graph.

Table 1. Node labels in the Design Grammar ontology schema (v6), grouped by Core band and the five graph layers.

Label	Graph	Key Property	Display Property
Class	OntoGraph	iri	label
DatatypeProperty	OntoGraph	iri	SWRL_label
ObjectProperty	OntoGraph	iri	label
Rule	Metagraph	Rule_Id	Rule_Id
Atom	Metagraph	Atom_Id	SWRL_label
Var	Metagraph	name	name
Literal	Metagraph	lex + datatype	lex
Builtin	OntoGraph	iri	label
RuleSet	Metagraph	name	name
Object	Core	objectId	label
Function	Core	—	label
Behaviour	Core	—	label
Structure	Core	—	label
Geometry	Core	—	label
Topology	Core	—	label
ObjectState	Core	stateId	label
DefState	Core	stateId	label
Session	Core	sessionId	mode
ValidationRun	ValidationGraph	runId	runId
ValidationEntity	ValidationGraph	dgEntityId	displayName

ObjectInstance	ValidationGraph	instanceId	displayName
GeoRef	ValidationGraph	geoRefId	geoRefId
Reasoner	ValidationGraph	name	name
IntegrationConfig	ValidationGraph	provider	provider
SpecNote	SpecGraph	noteId	title
SpecTag	SpecGraph	tagName	tagName
SpecClass	SpecGraph	name	name
Algorithm	ComputationGraph	algorithmName	algorithmName
Procedure	ComputationGraph	procedureName	procedureName
Pattern	ComputationGraph	patternName	patternName
Parameter	ComputationGraph	parameterName	parameterDisplayName
Interface	ComputationGraph	interfaceName	interfaceName

Table 2. Relationship types in the Design Grammar ontology schema (v6), including the cross-layer bridges.

Relationship	**Source → Target**	**Description**
HAS_BODY	Rule → Atom	Links a rule to its antecedent (body) atoms, ordered by pos.
HAS_HEAD	Rule → Atom	Links a rule to its consequent (head) atom.
REFERS_TO	Atom → OntoGraph entity	Maps an atom to the ontology term it instantiates (Class / DatatypeProperty / ObjectProperty / Builtin).
ARG	Atom → Var / Literal	Connects an atom to its argument variables and literals, ordered by pos.
VALIDATES	RuleSet → ObjectInstance	Cross-layer bridge linking the evaluated rule set to the object instances a validation run checks (Metagraph → ValidationGraph).
HAS_ENTITY	ValidationRun → ValidationEntity	Links a validation run to its per-object result entities.
VALIDATES_INSTANCE	ValidationEntity → ObjectInstance	Identifies the object instance a result entity concerns.
REFERS_TO_GEOMETRY	ValidationEntity → GeoRef	Bridges a result entity to the external geometry handle it concerns.
HAS_OBJECT_STATE	ObjectInstance → ObjectState	Binds an object instance to its captured variable-binding state.
DEFINES_FUNCTION	Object → Function	Associates a design object with the

		design function (intent) it fulfils.
HAS_ALGORITHM	Object → Algorithm	Links a design object to the parametric algorithm (definition) that generates it.
HAS_PROCEDURE	Algorithm → Procedure	Decomposes an algorithm into ordered procedures (clusters), ordered by procedureOrder.
HAS_PATTERN	Procedure → Pattern	Links a procedure to its constituent component patterns.
HAS_PARAMETER	Pattern → Parameter	Connects a pattern to its input and output parameters via typed interfaces.
ATTRIBUTE_OF	Atom → Parameter	Cross-layer bridge linking a rule atom to the ComputationGraph parameter it constrains (Metagraph → ComputationGraph).
TAGGED_WITH	SpecNote → SpecTag	Connects a knowledge note to the tags that classify it.

3.4 Modular Packaging: Core Ontology and Alignment Extensions

The schema is delivered as four OWL modules that follow the W3C core-plus-extension pattern used by vocabularies such as SOSA/SSN and DCAT. The core ontology contains only design-grammar-native content - the Core band and the five layer hubs with their classes, properties, and internal axioms - and declares no external imports, so it loads cleanly, reasons quickly, and stays vendor-neutral: no specific geometry, viewer, or modelling product is named anywhere in the core. Three extension modules add interoperability on top without modifying the core. The standards extension imports the core together with the W3C and OGC vocabularies (SWRL, PROV-O, SOSA, SHACL, SKOS, Dublin Core, and GeoSPARQL) and carries the cross-vocabulary alignment axioms described in Sections 3.1–3.3. The BOT extension maps the Core topology concept onto the Building Topology Ontology's building/storey/space decomposition. The Topologic extension bridges the geometric output of the ComputationGraph to Topologic's non-manifold spatial hierarchy (cluster, cell-complex, cell, shell, face, wire, edge, and vertex).

This separation keeps each concern independently maintainable and lets a consumer load only what is needed: the lean core for schema editing and rule authoring, or a given extension when reasoning across vocabularies or integrating with standards-aware tooling. Because the alignment axioms never live in the core, the ontology avoids the "ontology pollution" that results from re-parenting external classes under local hubs, and downstream systems that import only the core are not obliged to resolve seven external vocabularies. Table 3 summarizes the four modules and their roles.

Table 3. The four OWL modules of the Design Grammar ontology (v6).

Module	**Imports**	**Contents and role**
DesignGrammar.owl (core)	(none)	Core band and the five layer hubs; vendor-neutral DG content with no external imports.
DesignGrammar-standards-extension.owl	core + SWRL, PROV-O, SOSA, SHACL, SKOS, DCTerms, GeoSPARQL	Cross-vocabulary alignment axioms for HermiT reasoning and standards-aware tooling.
DesignGrammar-BOT-extension.owl	core + BOT	Maps the Core topology concept onto BOT building/storey/space decomposition.

DesignGrammar-Topologic-extension.owl	core	Bridges ComputationGraph geometry to Topologic non-manifold spatial hierarchy.

3.5 Mapping Parametric Definitions onto the ComputationGraph

The ComputationGraph is designed so that the parametric definitions architects already build in visual-programming environments can be expressed directly in the ontology, without translation loss. Grasshopper - the de facto authoring environment for which the system is built - organizes a definition as a canvas of wired components that may be grouped into named clusters. This structure maps one-to-one onto the ComputationGraph hierarchy: a Grasshopper definition is an Algorithm, each named cluster is a Procedure (ordered by procedureOrder), and each component within a cluster is a Pattern. The number sliders, panels, and value inputs that drive the definition become Parameter nodes, typed as constant, variable, or emergent and exposed through Input and Output Interface nodes; the wires between components are recorded by the paramLink relationship, and the geometry a component hosts by patternHostTo.

Figure 1 illustrates the mapping for a structural-frame definition. The whole canvas, labelled OBJECT - FRAME / 1_ALGORITHM, is the Algorithm that generates the frame Object; in Function–Behaviour–Structure terms it is the Behaviour that turns the design Function (a load-bearing frame) into a Structure (the frame geometry). Within it, two named clusters - 11_Proc - 2D Truss Configuration and 12_Proc - 2D Footer Configuration - are Procedures, and the components inside each cluster are Patterns. The sliders that set, for example, the truss height or the footer spacing are variable Parameters; quantities computed downstream from them, such as a derived member length, are emergent Parameters. The geometry produced (lower left) is the Structure, bound to the Object through the design state captured for a validation run.

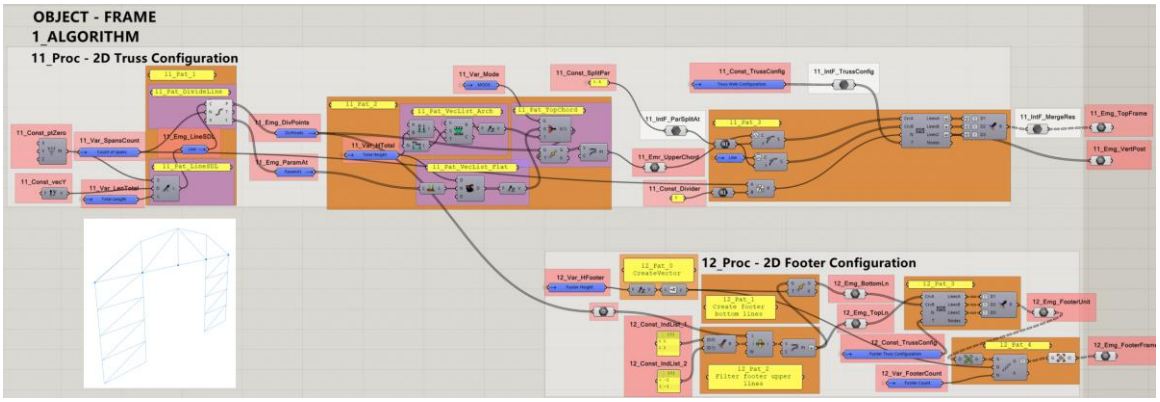

Figure 1. Mapping of a Grasshopper parametric definition (OBJECT - FRAME) onto the ComputationGraph: the definition is an Algorithm, named clusters are Procedures, components are Patterns, and sliders and computed values are Parameters.

The value of this mapping is that it makes a design rule and the parametric definition that must satisfy it mutually addressable. The cross-layer attributeOf bridge connects a Metagraph atom - for instance, the variable bound by swrlb:greaterThan in a maximum-height rule - to the specific ComputationGraph Parameter that produces the value being checked. A rule can therefore be traced back to the exact slider or computed quantity it constrains, and, conversely, a parameter can report which rules govern it. This two-way correspondence is the conceptual basis for the bidirectional translation between design grammars and parametric definitions that the companion work develops: rules authored in natural language can be projected onto the parameters of a live definition, and changes to a definition can be checked against the rules attached to its parameters.

3.6 Project Isolation and Knowledge Sharing

All nodes in both layers carry a project property that scopes them to a named design project, allowing a single graph to host multiple concurrent projects with logical isolation while still permitting cross-project queries. This design lets the ontology accumulate knowledge and reuse lessons learned across a shared stakeholder memory, rather than fragmenting it across isolated databases. The isolation strategy avoids the operational overhead of

per-project databases and is acceptable for pilot-scale use; multi-tenant production scenarios may require sharding or per-project databases, as discussed in Section 5.3.

4. ONTOLOGY EVALUATION THROUGH USE CASES

Because this paper concerns the ontology rather than a deployed system, evaluation is framed through three formalized use cases derived from the competency questions of Section 2.2. Each use case states the design question, identifies the ontology constructs that answer it, and indicates the intended future application. The use cases are illustrative of applicability potential; their quantitative, system-level evaluation is reserved for subsequent work.

4.1 Use Case 1: Natural-language design-rule communication and compliance checking

Question. Does a given building or space comply with a stated regulatory or project-specific constraint, expressed in natural language?

Ontology support. A natural-language rule such as “maximum building height is 75 m” corresponds to a single Rule node whose body atoms (ClassAtom, DataPropertyAtom, BuiltinAtom) encode the violating condition and whose head asserts a boolean violation flag - Building(?b) ∧ hasHeightM(?b, ?h) ∧ swrlb:greaterThan(?h, 75.0) → violatesMaxHeight(?b, true). Because each constraint is an addressable subgraph, rules can be inspected, audited, and communicated unambiguously among stakeholders, independently of any particular checking tool.

Future application. Machine-executable compliance checking against BIM geometry and natural-language authoring of rules; the encoding pipeline that converts natural language into this structure is detailed in the companion paper.

4.2 Use Case 2: Mixed-use master-plan design-state validation

Question. Does the current design state of a mixed-use master plan satisfy the planning constraints defined for each zone - for example, maximum building height per zone, minimum street-wall height, and maximum floor-plate area?

Ontology support. Each planning constraint maps to a separate Rule node following the rule-separation principle, and range constraints decompose into lower- and upper-bound rules. Project-scoped isolation lets several zones and design scenarios coexist within one graph, while the violation-pattern semantics makes the absence of violations a positive statement of compliance, provided the graph has been fully populated with the relevant model objects.

Future application. Longitudinal comparison of design states across iterative development cycles, supporting validation of how a scheme evolves through the design lifecycle.

4.3 Use Case 3: Encoded spatial-programme implementation in parametric layout generation

Question. Can a design space of plan configurations be generated that satisfies a spatial programme and its constraints - for example, minimum bedroom area (9 m²), minimum bathroom area (4 m²), minimum kitchen width (2.4 m), and at least one natural-light source per habitable room?

Ontology support. Spatial-programme requirements are expressed as constraints over domain classes (Bedroom, Bathroom, Kitchen), their datatype properties (area, width), and object properties (adjacency, hasWindow). The same constraint vocabulary that validates a model can also constrain a generative process, turning the ontology from a checker into a specification of the feasible design space.

Future application. Specification-driven parametric plan-layout generation, in which the ontology defines the feasible design space for downstream generative tools.

5. DISCUSSION

5.1 Expressiveness and Limitations of the SWRL Encoding

The current ontology encodes a subset of OWL 2 + SWRL expressiveness sufficient for quantitative threshold constraints (minimum, maximum, range, equality, cardinality). The schema supports three atom types - ClassAtom, DataPropertyAtom, and BuiltinAtom - which together cover the rule patterns illustrated in the use cases. It does not yet support disjunctive rules (A OR B), complex class expressions (intersection, union), or temporal constraints, which are common in building energy codes and fire-safety regulations. Extending the schema to accommodate these patterns is a priority for future work, likely through the addition of a Connector node type representing logical operators between atoms.

The violation-pattern semantics operates under an open-world assumption: if no Building node exists in the graph, a height rule will not fire, irrespective of whether the constraint is satisfied. This may be counterintuitive for practitioners expecting closed-world compliance semantics. The intended mitigation is to pre-populate the graph with all model objects before rule evaluation, so that the absence of a violation genuinely indicates compliance rather than an empty graph.

5.2 Positioning Relative to Related Approaches

Compared with IfcOWL + SPARQL-based compliance checkers (Pauwels et al., 2017), the proposed ontology does not require a full IfcOWL serialization of the BIM model, reducing the data-transformation burden on the authoring-tool side. Compared with rule systems built on proprietary languages, it stores rules in an open, queryable graph format that can be inspected and audited independently of the checking tool. Its distinguishing feature is methodological: the ontology is developed through explicit specification, conceptualization, formalization, and implementation, and through systematic reuse of FBS, DCM, BOT, and Topologic, rather than authored ad hoc for a single study. It also provides the ontological core on which the broader framework of Ermolenko et al. (2026) rests.

5.3 Scalability and Reuse Considerations

Project isolation via a property filter within a single graph is adequate for pilot-scale use of the ontology but should be revisited for multi-tenant settings, where sharding or per-project databases may be preferable. Because the schema decouples the ontology vocabulary from any specific BIM serialization format, it is positioned for reuse and alignment across systems; scaling to city-scale models or national regulatory libraries will require query-indexing strategies and is left for future work.

6. CONCLUSION

This paper has presented the development of an ontology for architectural design grammars, structured as typed SWRL rule constructs within a property graph. Following a systematic ontology-development methodology - specification, conceptualization, formalization, and implementation - and reusing established design and topology ontologies (FBS, DCM, BOT, Topologic), the work yields a five-layer schema organized beneath a cross-cutting, FBS-grounded Core band: an OntoGraph for ontological vocabulary, a Metagraph for rule decomposition, a ValidationGraph for execution and results, a SpecGraph for shared knowledge, and a ComputationGraph for the parametric design definition. A single cross-layer bridge links rule atoms to the parameters of the parametric definition, and a worked example shows how a Grasshopper definition maps directly onto the ComputationGraph. Delivered as a vendor-neutral core ontology with three optional alignment extensions (standards, BOT, and Topologic), the schema provides a modular, auditable, and technology-agnostic representation of design rules that can be evaluated directly against BIM geometry, with a violation-pattern semantics and a rule-separation principle that improve traceability and auditability.

Three formalized use cases - natural-language design-rule communication, mixed-use master-plan design-state validation, and spatial-programme-driven parametric layout generation - illustrate the ontology's applicability potential across checking, interaction, and generation. By concentrating on the ontology rather than on a particular software stack, the schema is positioned for reuse and alignment across systems.

Future work will (i) extend the schema to support disjunctive and temporal constraints, likely via a Connector node type for logical operators; (ii) align the ontology with IfcOWL and the buildingSMART Information Delivery Specification (IDS) to improve interoperability with standard exchange workflows; and (iii) operationalize the ontology within a complete natural-language authoring-and-validation framework, which is the subject of a companion paper. The ontology and supporting artefacts will be released as open source to support community adoption and comparative evaluation.

7. ACKNOWLEDGEMENTS

This work is supported by national funds through FCT – Foundation for Science and Technology under grant agreement 2024.03763.BD attributed to the first author, and by the "R2U Technologies | modular systems" project (ref.: C644876810-00000019) funded by PRR – Plano de Recuperação e Resiliência – and by European Funds Next Generation EU. Additional support was provided through the Multiannual Funding of Lab2PT (UID/04509), FCT R&D Unit ISISE (UID/4029/2025; UID/PRR/04029/2025), and Associate Laboratory ARISE (LA/P/0112/2020).

8. REFERENCES

Beirão, J. N., Duarte, J. P., & Stouffs, R. (2011). Creating Lisbon: Urban design with a city grammar. In Proceedings of the 2011 Symposium on Simulation for Architecture and Urban Design.

Dorst, K. (2011). The core of 'design thinking' and its application. Design Studies, 32(6), 521–532.

Dorst, K., & Cross, N. (2001). Creativity in the design process: Co-evolution of problem–solution. Design Studies, 22(5), 425–437.

Duarte, J. P. (2005). Towards the mass customization of housing: The grammar of Siza's Malagueira houses. Environment and Planning B: Planning and Design, 32(3), 347–380. https://doi.org/10.1068/b3144

Eastman, C., Teicholz, P., Sacks, R., & Liston, K. (2009). BIM handbook: A guide to building information modeling for owners, managers, designers, engineers and contractors. Wiley.

Ermolenko, E., Figueiredo, B., & Azenha, M. (2026). Management of the design knowledge core in a cross-platform environment of the BIM project. In Proceedings of the ptBIM 2026 Conference. University of Minho.

Fu, K., Murphy, J., Yang, M., Otto, K., Jensen, D., & Wood, K. (2015). Design-by-analogy: Experimental evaluation of a functional analogy search methodology for concept generation improvement. Research in Engineering Design, 26(1), 77–95.

Gero, J. S. (1990). Design prototypes: A knowledge representation schema for design. AI Magazine, 11(4), 26–36.

Gero, J. S. (1992). What's in a design case and how it can be used. In AID'92 Workshop on Case-Based Design Systems (pp. 27–28).

Gero, J. S., & Kannengiesser, U. (2004). The situated function–behaviour–structure framework. Design Studies, 25(4), 373–391.

Jabi, W. (2021). Topologic: Tools for nonmanifold topology in architectural design. https://topologic.app

Lin, C.-J. (2021). Design Criteria Modelling: Use of ontology-based algorithmic modelling. Automation in Construction.

Mubarak, K. (2004). Case-based reasoning for design composition in architecture. Carnegie Mellon University, Pittsburgh.

Noy, N. F., & McGuinness, D. L. (2001). Ontology development 101: A guide to creating your first ontology. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.

Pauwels, P., & Terkaj, W. (2016). EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology. Automation in Construction, 63, 100–133. https://doi.org/10.1016/j.autcon.2015.12.003

Pauwels, P., Zhang, S., & Lee, Y. C. (2017). Semantic web technologies in AEC industry: A literature overview. Automation in Construction, 83, 145–165. https://doi.org/10.1016/j.autcon.2017.03.009

Poon, J., & Maher, M. L. (1997). Co-evolution and emergence in design. Artificial Intelligence in Engineering, 11(3), 319–327.

Rasmussen, M. H., Lefrançois, M., Schneider, G. F., & Pauwels, P. (2020). BOT: The building topology ontology of the W3C linked building data community group. Semantic Web, 12(1), 143–161. https://doi.org/10.3233/SW-200385

Rasmussen, M. H., Pauwels, P., Lefrançois, M., & Schneider, G. F. (2017). Building topology ontology. In Proceedings of the 14th International Conference on Computing in Civil and Building Engineering.

Sack, H. (2019). Knowledge engineering with semantic web technologies: Ontological engineering [MOOC]. openHPI.

Sacks, R., Brilakis, I., Pikas, E., Xie, H. S., & Girolami, M. (2020). Construction with digital twin information systems. Data-Centric Engineering, 1, e14. https://doi.org/10.1017/dce.2020.16

Schön, D. A. (1992). Designing as reflective conversation with the materials of a design situation. Research in Engineering Design, 3(3), 131–147.

Stiny, G., & Gips, J. (1972). Shape grammars and the generative specification of painting and sculpture. In C. V. Freiman (Ed.), Information Processing 71 (pp. 1460–1465). North-Holland.

Zhou, Y., Braun, A., & Borrmann, A. (2022). Knowledge graph-based automated code compliance checking for building design. In Proceedings of the European Conference on Computing in Construction. https://doi.org/10.35490/EC3.2022.188